

Desert Car Runner



Submitted by:

M.Sharjeel 2025-CS-1205

Section C

Supervised by:

Sir.Talha Mubashir

Department of Computer Science

University of Engineering and Technology

Lahore Pakistan

Short Description :

Desert Rally Race is a console-based racing game developed in C++. The player controls a car moving through a desert road with three lanes, avoiding obstacles, Other car (X), Rock (R) and Cactus [A] collecting fuel [F] , and boosting speed. The speed increases by 10 per 5km. Making the car fast which gradually increases difficulty level. The game tracks distance, fuel, health, and score, ending when the car crashes, fuel runs out, or the player quits.

Users of Game :

There is only a single Player in this Game.

Functional requirements :

- Display a 3-lane road using ASCII graphics in the console.
- Allow player movement using A,D,I,K keys (left, right, up, down).
- Generate random obstacles:
 - Cactus, rock, other cars, and fuel cans.
- Implement collision detection with obstacles.
- Manage fuel, health, speed, score, and distance.
- Enable boost mode using the space bar (fuel-based).
- Show real-time HUD (score, speed, fuel, health).
- End the game when fuel or health reaches zero.
- Save final score and statistics to a file.
- Allow restart or quit after game over.

Wire Frames :

Welcome screen.

```
=====
=      DESERT RALLY RACE - HOW TO PLAY      =
=====

Welcome to Desert Rally Race.
OBJECTIVE:
Drive as far as you can without crashing!

CONTROLS:
A / D      = Move Left / Right
W          = Boost (costs fuel)
Q          = Quit Game

OBSTACLES:
X Cactus   = -1 Health
R Rock     = -2 Health
[A] Other Car = -3 Health

POWER-UPS:
[F] Fuel Can = +25% Fuel

SCORING:
- Points for distance traveled
- Bonus for fuel collected
- Penalty for collisions

GOOD LUCK!
Press any key to start...|
```

Player interface:

```

===== DESERT RALLY RACE =====

|| || || [A] || ===== | ===== | ===== SCORE: 675
|| || || R   ||                                     SPEED: 70 km/h
|| || ||     ||                                     DISTANCE: 5.25 km
|| || || X   || ===== | ===== | =====
|| || || X   || FUEL: [===== ] 63%
|| || ||     || HEALTH: [o] [o] [o] [o] [ ]
|| || ||     || ===== | ===== | =====
|| || ||     || CONTROLS: left/RightKey=Move, Up/downkey=Up/Down,
|| || ||     || ===== | ===== | =====
|| || || [F] || ===== | ===== | =====
|| || ||     ||
|| || || X   || ===== | ===== | =====
|| || || X   || ===== | ===== | =====
|| || || R   || ===== | ===== | =====
|| || ||     || ===== | ===== | =====
|| || ||     ||

```

Complete Code:

```
#include <iostream>
```

```
#include <conio.h>
```

```
#include <windows.h>
```

```
#include <fstream>
```

```
#include <cstdlib>    // This library is used for rand() and srand() for  
random number or obstacle generator.
```

```
#include <ctime>    // This library can be used for time
```

```
using namespace std;
```

```
const int ROAD_WIDTH = 3;    // 3 lanes: Left(0), Center(1),  
Right(2)
```

```
const int ROAD_LENGTH = 20;  // No of rows of road to display
```

```
const int MAX_FUEL = 100;
```

```
const int MAX_HEALTH = 5;
```

```
int playerLane = 1;
```

```
int playerRow;
```

```
int fuel = MAX_FUEL;
```

```
int health = MAX_HEALTH;
```

```
int score = 0;
```

```
int totaldistance = 0;
```

```
int speed = 60;
```

```
int boostTimer = 0;
```

```
bool gameOver = false;
```

```
// Road grid: [lane][position] - 0=empty, 1=cactus, 2=rock, 3=fuel,  
4=car
```

```
int road[ROAD_WIDTH][ROAD_LENGTH];
```

```
void gotoXY(int x, int y) {
```

```
    COORD c = { (SHORT)x, (SHORT)y };
```

```
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HAND  
LE), c);
```

```
}
```

```
void clearScreen() {
```

```
    gotoXY(0, 0);
```

```
    for (int i = 0; i < 50; i++) {
```

```
        cout << "                                     \n";
```

```
    }
```

```
    gotoXY(0, 0);
```

```
}
```

```
bool showGameOver();
```

```
void showInstructions();
```

```
void setup() {  
    playerRow = ROAD_LENGTH - 3;  
  
    for (int lane = 0; lane < ROAD_WIDTH; lane++) {  
        for (int pos = 0; pos < ROAD_LENGTH; pos++) {  
            road[lane][pos] = 0;  
        }  
    }  
}  
  
void draw() {  
    int roadStartX = 5;  
    int roadStartY = 4;  
    int hudStartX = 50;  
    const int LANE_WIDTH = 5;  
  
    gotoXY(roadStartX, roadStartY - 3);  
    cout << "\033[33m      ===== DESERT RALLY RACE  
===== \033[0m " << endl;  
  
    // Draw each row of the road  
    for (int pos = 0; pos < ROAD_LENGTH; pos++) {  
        gotoXY(roadStartX, roadStartY + pos);  
  
        cout << "||";
```

```
for (int lane = 0; lane < ROAD_WIDTH; lane++) {

    if (pos == playerRow && lane == playerLane) {
        cout << "[==] ";
    } else {
        switch(road[lane][pos]) {
            case 0: cout << "   "; break;
            case 1: cout << " X "; break;
            case 2: cout << " R "; break;
            case 3: cout << " [F] "; break;
            case 4: cout << " [A] "; break;
        }
    }
    cout << "||";
}

if (pos % 3 == 0)
    cout << "=====|=====|=====";

gotoXY(hudStartX, roadStartY + pos);
if (pos == 0) cout << "\033[32mSCORE:\033[0m " << score;
```

```
    if (pos == 1) cout << "\033[32mSPEED\033[0m: " << speed <<
    " km/h";

    if (pos == 2) cout << "\033[32mDISTANCE:\033[0m " <<
    totaldistance / 1000.0 << " km";

    if (pos == 4) {
        cout << "FUEL: [";
        for (int i = 0; i < 10; i++)
            cout << (i < fuel / 10 ? "=" : " ");
        cout << "]" << fuel << "%";
    }

    if (pos == 5) {
        cout << "HEALTH: ";
        for (int i = 0; i < health; i++) cout << "[o] ";
        for (int i = health; i < MAX_HEALTH; i++) cout << "[ ] ";
    }

    if (pos == 7) {
        cout << "\033[34mCONTROLS: left/RightKey=Move,
    Up/downkey=Up/Down, Space=Boost, Q=Quit\033[0m";
    }

    if (pos == 8 && fuel < 30) {
        cout << "\033[31mLOW FUEL! Find\033[0m [F]";
    } else{
        cout << "                ";
```

```
    }  
}  
}
```

```
void input() {  
    if (_kbhit()) {  
        char key = _getch();  
  
        switch (tolower(key)) {  
  
            case 'a':  
                if (playerLane > 0)  
                    playerLane--;  
                break;  
  
            case 'd':  
                if (playerLane < ROAD_WIDTH - 1) {  
                    playerLane++;  
                    Sleep(45);  
                }  
                break;  
  
            case 'i':
```

```
        if (playerRow > 1)
            playerRow--;
        break;

    case 'k':
        if (playerRow < ROAD_LENGTH - 2)
            playerRow++;
        break;

    case 'q':
        gameOver = true;
        break;
    }
}

void generateObstacle() {

    int lane = rand() % ROAD_WIDTH;

    //To avoid generating on existing obstacle
    if (road[lane][0] != 0) return;

    // (weighted probabilities)
```

```
int obstacleType = rand() % 100;

if (obstacleType < 40) {      // 40% chance: Cactus
    road[lane][0] = 1;
} else if (obstacleType < 60) { // 30% chance: Rock
    road[lane][0] = 2;
} else if (obstacleType < 80) { // 25% chance: Fuel
    road[lane][0] = 3;
} else {                      // 15% chance: Other car
    road[lane][0] = 4;
}
}

void moveObstacles() {

    for (int lane = 0; lane < ROAD_WIDTH; lane++) {
        for (int pos = ROAD_LENGTH - 1; pos > 0; pos--) {
            road[lane][pos] = road[lane][pos - 1];
        }
        road[lane][0] = 0; // Clear top row
    }
}
```

```
void checkCollisions() {  
  
    int objectInFront = road[playerLane][playerRow];  
  
    if (objectInFront != 0) {  
        switch(objectInFront) {  
            case 1: // Cactus  
                health--;  
                score -= 50;  
                cout << "\a";  
                break;  
  
            case 2: // Rock  
                health -= 2;  
                score -= 100;  
                cout << "\a\a";  
                break;  
  
            case 3: // Fuel  
                fuel += 25;  
                if (fuel > MAX_FUEL) fuel = MAX_FUEL;  
                score += 100;  
                break;
```

```
        case 4: // Other car
            health -= 3;
            score -= 200;
            cout << "\a\a\a";
            break;
    }
    road[playerLane][playerRow] = 0;

    if (health <= 0) {
        health = 0;
        gameOver = true;
    }
}

void update() {

    totaldistance += speed;

    score += speed / 10;
```

```
static int nextSpinc = 5000;  
int maxSpeed = (boostTimer > 0) ? 140 : 120;
```

```
if(totaldistance >= nextSpinc){  
    speed += 10;  
    if(speed > maxSpeed)  
        speed = maxSpeed;  
nextSpinc += 5000;  
}
```

```
fuel -= 1;  
if (fuel <= 0) {  
    fuel = 0;  
    gameOver = true;  
}
```

```
if (boostTimer > 0) {  
    boostTimer--;  
    if (boostTimer == 0) {  
        speed -= 20;
```

```
        if (speed < 60) speed = 60;
    }
}

moveObstacles();
// Generate new obstacles randomly
if (rand() % 100 < 30) {
    generateObstacle();
}
checkCollisions();
}

void resetGame() {
    playerLane = 1;
    fuel = MAX_FUEL;
    health = MAX_HEALTH;
    score = 0;
    totaldistance = 0;
    speed = 60;
    boostTimer = 0;
    gameOver = false;
    // Clear road
    for (int lane = 0; lane < ROAD_WIDTH; lane++) {
        for (int pos = 0; pos < ROAD_LENGTH; pos++) {
```

```
        road[lane][pos] = 0;
    }
}
}

int main() {
    srand(time(0));
    showInstructions();
    do{
        system("cls");
        setup();
        while (!gameOver) {
            draw();
            input();
            update();
            Sleep(50);
        }
    } while(showGameOver());
}

void showInstructions() {
    clearScreen();
    cout <<
    "    =====
    " << endl;
```

```
cout << "    = DESERT RALLY RACE - HOW TO  
PLAY    =    " << endl;  
  
cout <<  
"    =====  
" << endl;  
  
cout << "\033[33mWelcome to Desert Rally Race.\033[0m" <<  
endl;  
  
cout << "\033[34mOBJECTIVE:\033[0m\n";  
cout << "Drive as far as you can without crashing!\n\n";  
  
  
cout << "\033[34mCONTROLS:\033[0m\n";  
cout << "A / D    = Move Left / Right\n";  
cout << "Q        = Quit Game\n\n";  
  
  
cout << "\033[34mOBSTACLES:\033[0m\n";  
cout << " X Cactus   = -1 Health\n";  
cout << " R Rock     = -2 Health\n";  
cout << "[A] Other Car = -3 Health\n\n";  
  
  
cout << "\033[34mPOWER-UPS:\033[0m\n";  
cout << "[F] Fuel Can = +25% Fuel\n\n";  
  
  
cout << "\033[34mSCORING:\033[0m\n";  
cout << "- Points for distance traveled\n";
```

```
    cout << "- Bonus for fuel collected\n";
    cout << "- Penalty for collisions\n\n";

    cout << "\033[32mGOOD LUCK!\033[0m " <<endl;
    cout << "Press any key to start...";
    _getch();
}

bool showGameOver() {
    clearScreen();

    int finalScore = score + (fuel * 2) + (totaldistance / 100);

    cout << "\n\n";
    cout <<
    "===== \n";
    cout << "          GAME OVER\n";
    cout <<
    "===== \n\n";

    if (fuel == 0) {
        cout << "  \033[31mYou ran out of fuel!\033[0m\n";
        cout << "  \033[31m[F] > (X)\n\n\033[0m";
    } else if (health == 0) {
        cout << "  \033[33mYou crashed your car!\033[0m\n";
        cout << "  \033[33m[==] > [R]\n\n\033[0m";
    }
}
```

```
} else {  
    cout << "    You quit the race.\n\n";  
}  
  
ofstream file("Score.txt", ios::app);  
if(!file.is_open()){  
    cout << "Cannot save score to file..\n";  
} else  
{  
file << "FINAL STATISTICS:\n";  
file << "-----\n";  
file << "Distance: " << totaldistance / 1000.0 << " km\n";  
file << "Max Speed: " << speed << " km/h\n";  
file << "Fuel Left: " << fuel << "%\n";  
file << "Health:  " << health << "/" << MAX_HEALTH <<  
"\n";  
file << "Base Score: " << score << "\n";  
file << "Fuel Bonus: +" << (fuel * 2) << "\n";  
file << "Dist Bonus: +" << (totaldistance / 100) << "\n";  
file << "-----\n";  
file << "TOTAL SCORE: " << finalScore << "\n\n";  
file.close();  
}
```

```
cout << "\033[32mFINAL STATISTICS:\033[0m\n";
cout << "-----\n";
cout << "Distance: " << totaldistance / 1000.0 << " km\n";
cout << "Max Speed: " << speed << " km/h\n";
cout << "Fuel Left: " << fuel << "%\n";
cout << "Health:  " << health << "/" << MAX_HEALTH <<
"\n";
cout << "Base Score: " << score << "\n";
cout << "Fuel Bonus: +" << (fuel * 2) << "\n";
cout << "Dist Bonus: +" << (totaldistance / 100) << "\n";
cout << "-----\n";
cout << "\033[32mTOTAL SCORE:\033[0m " << finalScore <<
"\n\n";
cout << "\033[32mPERFORMANCE:\033[0m ";
    if (finalScore > 5000) cout << " \033[33mRACE
CHAMPION!\033[0m";
    else if (finalScore > 3000) cout << " \033[34mEXCELLENT
DRIVER!\033[0m";
    else if (finalScore > 1000) cout << " \033[34mGOOD
EFFORT!\033[0m";
    else cout << "KEEP PRACTICING!";
cout << "\n\n";
cout << "[P] Play Again  [Q] Quit\n";
char choice;
while(true){
```

```
if (_kbhit()) {  
    choice = _getch();  
    choice = tolower(choice);  
    if (choice == 'p') {  
        // Reset game and restart  
        resetGame();  
        setup();  
        return true; // Exit this function  
    } else if ((choice) == 'q') {  
        return false;  
    }  
}  
Sleep(50);  
}
```

Weakness in Application:

- Windows-only (not cross-platform).
- Console graphics limit visual quality. Graphics should be added to make it realistic.
- Emogies should be used instead of text formats for obstacles and fuel.

Future Directions:

In Future we can add multiple levels with conditions and the game can be made more realistic and engaging by adding multiplayer to game.

